Major : Software Engineering

Risk Management (ISO 27001 / 27017) for a Secure AWS Architecture and DevSecOps Pipeline

Prepared by :
TAIK Kawtar
Jamyl Hanane
OUAZRI Khaoula
EL ALOUAN Wisal

Supervised by :
Mrs. SADKI Souad

Academic year : 2025/2026

Contents

Chapter 1
Risk Management Process (ISO 27001)

This reports details the risk management process applied to the "Secure Cloud & AI Architecture for a Research Website" project , following the six-step framework outlined in the project specifications and aligning with the ISO/IEC 27001 standard, supplemented by controls from ISO/IEC 27017 (Cloud Security)and ISO/IEC 27034 (Application Security).

1.1 Asset Identification

The first step is to identify and catalog all critical assets within the system to establish a comprehensive inventory for risk management. This systematic documentation process forms the foundation for understanding what resources require protection and enables effective risk assessment throughout the organization.

Assets to identify:

- Web application and database infrastructure.
- AI components (models, datasets, APIs).
- User data and research statistics.
- DevSecOps pipeline and automation tools.

The following table presents the inventory of identified assets, their classification, and an evaluation of their criticality based on Confidentiality (C), Integrity (I), and Availability (A):

Category	Asset	C	I	A	Justification (for High ratings)
Data Assets					
	MySQL Database (Global statistics)	M	H	H	**I/A:** The integrity and availability of this data are the core mission of the website.
	Researcher-uploaded datasets (Future)	H	H	M	**C/I:** Potentially sensitive, private research data. Must not be leaked or altered.
	CloudTrail Logs	H	H	M	**C/I:** Contains audit trail for security investigations. Log integrity is crucial for forensics.

Continued on next page...

Category	Asset	C	I	A	Justification (for High ratings)
	RDS Backup Snapshots	H	H	M	**C/I:** Encrypted backups contain full database copies. Integrity critical for disaster recovery.
Infrastructure Assets					
	VPC and Subnets	M	H	H	**I/A:** Network segmentation integrity prevents unauthorized access. Availability ensures service continuity.
	EC2 Web Server Instances	M	H	H	**I/A:** Code integrity prevents malware injection. Availability required for user access.
	RDS MySQL Instance	M	H	H	**I/A:** Database service must maintain data integrity and remain available for queries.
	Application Load Balancer (ALB)	L	M	H	**A:** Single point of entry; unavailability blocks all access to the website.
	NAT Gateway	L	M	H	**A:** Required for instances in private subnets to reach internet for updates.
	Security Groups	M	H	M	**I:** Misconfiguration can expose database or allow unauthorized access.
	Internet Gateway	L	M	H	**A:** Gateway failure prevents all external access to the platform.
Application Assets					
	PHP Web Application Code	M	H	H	**I/A:** Code integrity prevents vulnerabilities (SQL injection, XSS). Availability serves researchers.
	Application Configuration Files	H	H	M	**C/I:** Contains database credentials and API keys. Modification can compromise security.
	SSL/TLS Certificates	M	H	M	**I:** Certificate integrity ensures encrypted connections and prevents MITM attacks.
	Session Data	H	M	M	**C:** Session tokens enable user impersonation if exposed.

Continued on next page...

Category	Asset	C	I	A	Justification (for High ratings)
AI Components (Future)					
	AI Models (Machine Learning)	H	H	H	**C/I/A:** Proprietary IP requiring protection. Model integrity prevents bias. Availability enables AI features.
	Training Datasets	H	H	M	**C/I:** Contains potentially sensitive research data. Poisoning training data causes model bias.
	AI Inference API	M	H	H	**I/A:** API integrity prevents prompt injection. Availability required for real-time predictions.
	Model Parameters and Weights	H	H	L	**C/I:** Represents intellectual property and research investment. Theft enables competitors.
Security & Access Control Assets					
	IAM Roles and Policies	H	H	H	**C/I/A:** Controls all access to AWS resources. Compromise enables full system breach.
	AWS KMS Encryption Keys	H	H	M	**C/I:** Protects data at rest. Key compromise exposes all encrypted data.
	Admin SSH Keys	H	M	L	**C:** Unauthorized access to SSH keys enables server compromise.
	Database Credentials	H	H	M	**C/I:** Direct database access credentials; exposure enables data breach.
	API Keys (Third-party services)	H	M	M	**C:** Leaked keys enable unauthorized use of paid services and data access.
DevSecOps Pipeline Assets					
	Source Code Repository (GitHub)	M	H	M	**I:** Repository integrity prevents malicious code injection into production.
	CI/CD Pipeline (CodePipeline)	M	H	H	**I/A:** Pipeline compromise deploys malicious code. Availability ensures continuous deployment.

Continued on next page...

Category	Asset	C	I	A	Justification (for High ratings)
	Infrastructure as Code (Terraform/CloudFormation)	M	H	M	**I:** IaC defines infrastructure; tampering creates security misconfigurations.
	Container Images (if applicable)	M	H	M	**I:** Image vulnerabilities or malware propagate to all deployed containers.
	Build Artifacts	M	H	L	**I:** Compromised artifacts introduce vulnerabilities into production deployments.
	Pipeline Credentials and Secrets	H	H	M	**C/I:** Access to pipeline secrets enables unauthorized deployments and infrastructure changes.
Monitoring & Logging Assets					
	CloudWatch Logs	M	H	M	**I:** Log integrity required for incident investigation and compliance auditing.
	VPC Flow Logs	M	H	L	**I:** Network traffic logs essential for security forensics and threat detection.
	GuardDuty Findings	H	H	M	**C/I:** Threat intelligence data guides security responses. Tampering hides attacks.
	Security Hub Reports	M	H	L	**I:** Centralized security posture visibility; altered reports mislead security teams.
Personnel & User Assets					
	Administrator Accounts	H	H	H	**C/I/A:** Full system control. Compromise enables complete infrastructure takeover.
	Developer Accounts	M	H	M	**I:** Access to code and deployment. Compromised accounts inject malicious code.
	Researcher User Accounts	M	M	M	**C:** User credentials enable unauthorized access to platform features.
	Service Accounts (Application)	H	H	H	**C/I/A:** Automated processes with elevated privileges. Compromise enables persistent access.

1.2 Threat Identification

The primary objective of this phase is to systematically identify and document all potential security threats that could compromise the research platform.

1.2.1 Threat Analysis

The following analysis presents the identified threats, categorized by system layer.

- **Infrastructure (Cloud):**
  1. **Misconfiguration of AWS Services:** (e.g., public S3 buckets, overly permissive Security Groups). Critical as it bypasses network defenses and could lead to data exposure
  2. **Compromised IAM Credentials:** Stolen keys can grant an attacker full administrative access, leading to total system compromise.
  3. **Denial of Service (DDoS) Attack:** A successful attack on the ALB or EC2 instances would violate Availability (A) for all users.
- **Application (Web):**
  1. **SQL Injection:** a poorly configured rule or novel attack vector could allow data exfiltration or modification (violating C and I).
  2. **Cross-Site Scripting (XSS):** Could be used to steal researcher session cookies or deface the site.
  3. **Vulnerable Dependencies:** Use of outdated PHP components or libraries could open known vulnerabilities.
- **Data & AI:**
  1. **Data Exfiltration:** Unauthorized access to the RDS database by an insider or an external attacker.
  2. **Prompt Injection (AI):** An attacker crafts inputs to make the AI assistant bypass its instructions, reveal sensitive data, or execute unintended actions.
  3. **Model Theft (AI):** Unauthorized copying of the trained AI model, representing a significant loss of intellectual property.
- **DevSecOps Pipeline:**
  1. **Pipeline Poisoning:** Malicious code injected into the Git repository or a build script , which is then automatically tested and deployed *securely* to production.

1.2.2 Specific Impact on AI Functionality

The identified threats pose specific risks to AI functionality through multiple attack vectors and exploitation mechanisms. Understanding how general system threats cascade to affect AI components is critical for implementing appropriate security controls that protect machine learning models, training data, and inference services.

Infrastructure Threats Compromising AI Assets:

- **Misconfiguration (T.1):** An improper AWS configuration, like a public S3 bucket, could expose AI models for theft. This also allows attackers to inject "poisoned" data into the training set, corrupting the model's behavior and leading to biased or inaccurate results.

- **Compromised Credentials (T.2):** Stolen AWS credentials grant direct, low-level access to AI assets, bypassing application-layer security. This allows attackers to steal models, poison training data, or even deploy backdoored models directly into production.

Application Layer Threats Affecting AI Operations:

- **Vulnerable Dependencies (T.6):** The AI relies on complex Python libraries (e.g., TensorFlow, PyTorch). A vulnerability in one of these dependencies could allow an attacker to manipulate the model's predictions, steal data during processing, or inject malicious code into the AI infrastructure.

Direct AI-Specific Threats:

- **Prompt Injection (T.8):** This is a direct attack using crafted text inputs. By telling the AI to "ignore previous instructions," an attacker can bypass its safety rules. This could be used to leak sensitive data, generate harmful content, or query connected databases—similar to an SQL injection, but using natural language.
- **Model Theft (T.9):** This attack targets the AI model as intellectual property. Attackers can either steal the model files directly from cloud storage or perform a "model extraction" attack by repeatedly querying the API to reconstruct how it works. The impact is a direct loss of competitive advantage and potential misuse of the stolen AI.

1.3 Risk Assessment

The objective of this phase is to assess the likelihood and impact of each identified threat to prioritize them for treatment. Risk scoring employs a qualitative matrix approach that balances ease of assessment with meaningful differentiation between threat severities.

1.3.1 Risk Calculation Methodology

Key Question: Risk Score Calculation

"For a given threat, how would you calculate its risk score?"

The methodology incorporates two fundamental dimensions that together determine overall risk exposure.

- **Likelihood (L):** Represents the probability of a threat materializing, rated on a scale from 1 to 5.
  - A rating of 1 (Rare) indicates threats occurring less than once per year, typically requiring sophisticated attacker capabilities or specific environmental conditions.
  - A rating of 3 (Possible) indicates threats with moderate probability occurring several times per year given current security posture.
  - A rating of 5 (Very Likely) indicates threats occurring multiple times per year or continuously, often through automated scanning or common attack vectors requiring minimal sophistication.
- **Impact (I):** Measures the potential damage to confidentiality, integrity, and availability if the threat succeeds, rated from 1 to 5.
  - A rating of 1 (Insignificant) indicates minimal disruption with immediate recovery and no data loss.

  - A rating of 3 (Moderate) indicates significant business disruption requiring manual recovery efforts with limited data loss.
  - A rating of 5 (Catastrophic) indicates complete system compromise, massive data breach, extended service outages, irreversible damage, or regulatory violations with severe financial and reputational consequences.

- **Risk Score Calculation:** The risk score is calculated as the product of likelihood and impact:

$$\textbf{Risk Score} = \textbf{Likelihood} \times \textbf{Impact}$$

This multiplicative approach ensures that threats with either high likelihood or high impact receive appropriate attention, while threats that are both unlikely and low-impact remain deprioritized.

1.3.2 Prioritized Threats for Immediate Treatment

Based on the methodology above, the threats requiring immediate (Critical/High) treatment are:

- **Compromised IAM Credentials (T.2):** (Likelihood: 3, Impact: 5, Score: 15 - High). Can lead to full system takeover.
- **SQL Injection (T.4):** (Likelihood: 4, Impact: 4, Score: 16 - High). A common and high-impact web attack.
- **Misconfiguration of AWS Services (T.1):** (Likelihood: 3, Impact: 5, Score: 15 - High). Easy to make a mistake, with critical consequences like data exposure
- **Prompt Injection (T.8):** (Likelihood: 4, Impact: 4, Score: 16 - High). A new attack vector, and the AI is a primary new feature.

1.4 Control Mapping (Annex A)

The control mapping phase identifies and assigns specific security controls that will mitigate the risks identified in previous phases. Controls are selected from three complementary standards: ISO 27001 Annex A provides the foundational information security controls, ISO 27017 adds cloud computing specific protections for the AWS environment, and ISO 27034 contributes application security controls for the web and AI components.

1.4.1 Control Application

The following controls from ISO 27001 Annex A, ISO 27017, and ISO 27034 have been selected to protect critical system assets based on their specific security requirements and threat exposure.

- **To Protect the Database (RDS):**
  - **A.9.1.2 (Network Access Control):** Placing RDS in private subnets and using Security Groups to restrict access
  - **A.10.1.1 (Cryptography):** Encrypting data at rest (via KMS) and in transit (via TLS).
  - **A.12.3.1 (Backup):** Enabling RDS automated backups and point-in-time recovery.
- **To Protect the Web Application (EC2):**
  - **A.12.6.1 (Vulnerability Management):** Regular scanning (integrated into DevSecOps).
  - **A.14.1.2 (Security in Development):** Applying AWS WAF for SQLi/XSS protection.
  - **ISO 27017 (Cloud Control):** Monitoring of cloud services; using CloudWatch and CloudTrail

- **To Protect the AI Models:**
  - **A.8.2.1 (Classification):** Classifying the model and training data as Confidential.
  - **A.9.4.1 (Access Control):** Using strict S3 bucket policies and IAM roles to limit who/what can access the model files.
  - **ISO 27034 (AppSec):** Security testing during development; specifically testing for prompt injection and data poisoning vulnerabilities

1.4.2 Enforcing Least Privilege

Least privilege is applied at every layer as part of a "Defense in Depth" strategy:

- **AWS Infrastructure:** Using specific IAM Roles for services (e.g., an EC2 role that can only connect to RDS, not administer it).
- **Network:** Using Security Groups to define explicit "allow" rules (e.g., only the ALB Security Group can access the EC2 instances on port 443).
- **Operating System:** Restricting administrative access to EC2 instances via AWS Systems Manager Session Manager instead of open SSH ports.
- **Application:** Creating different roles (e.g., "Researcher" vs. "Admin") within the PHP application itself.

1.4.3 DevSecOps Pipeline Compliance

The DevSecOps pipeline represents a critical attack vector, as compromised development and deployment processes can inject malicious code directly into production environments, bypassing traditional security controls.

- **A.14.2.1 (Secure Development Policy):** The policy is the implementation of the DevSecOps pipeline itself.
- **A.14.2.5 (Secure System Engineering):** Using Infrastructure as Code (IaC) to create repeatable, auditable environments .
- **A.14.2.8 (System Security Testing):** Automating SAST, DAST, and Dependency Scanning within the pipeline.
- **A.12.6.1 (Vulnerability Management):** Automatically failing the build if critical vulnerabilities are found

1.5 Risk Treatment

1.5.1 Securing Web App and Database

The "After" architecture design directly treats these risks:

- **Web App Access:** Researchers access the app via an ALB protected by AWS WAF. The EC2 instances themselves are in private subnets, inaccessible from the public internet.
- **Database Access:** The MySQL database is moved to Amazon RDS and placed in its own private subnets. It is not publicly accessible.
- **Connecting Flow:** The only connection allowed to the database is from the EC2 instances' Security Group , enforcing the principle of least privilege.

1.5.2 Preventing AI-Specific Risks

- **Prompt Injection:** Implement strict input sanitization and output encoding. Use parameterized queries to the database (even if via the AI). Instruct the AI model (via its "system prompt") to never execute user instructions that contradict its core purpose.
- **Model Theft:** Use AWS KMS to encrypt the model at rest. Use fine-grained IAM roles and S3 bucket policies to ensure only the production AI service can read the model file.
- **Hallucinations:** Implement a Retrieval-Augmented Generation (RAG) pattern. The AI does not answer from its internal knowledge; it first queries the (trusted) MySQL database based on the user's question, and then synthesizes an answer based only on the results returned from the database.

1.5.3 Monitoring and Incident Response

- **Monitoring:** Enable AWS CloudTrail (for API call auditing), CloudWatch Logs (for application/system logs) , and VPC Flow Logs (for network traffic analysis)
- **Alerting:** Create CloudWatch Alarms based on specific metrics (e.g., high CPU, >5 failed logins) or log events (e.g., "Access Denied" to S3 model bucket).
- **Response:** Configure alarms to trigger Amazon Simple Notification Service (SNS) topics, which can send emails or trigger automated remediation via AWS Lambda.

1.6 Monitoring and Review

The primary objective of this phase is to design monitoring and auditing processes for the entire system to ensure the risk management framework is a continuous cycle.

1.6.1 Monitoring Toolset

- **Infrastructure:** AWS CloudTrail (for all API activity), AWS Config (for compliance checks and configuration drift detection) , CloudWatch (for metrics and logs) , and VPC Flow Logs (for network analysis).
- **Application:** CloudWatch (for application-level logs) and the AWS WAF Dashboard (for monitoring blocked attacks like SQLi/XSS).
- **AI Models:** CloudWatch to monitor the AI's API logs. This is critical for detecting anomalies like a sudden spike in complex queries (potential DoS) or repeated, probing queries (potential prompt injection attempts).

1.6.2 Updating the Risk Assessment

The risk assessment is a living document, not a one-time task. It must be updated in response to specific triggers:

1. **Trigger (Major Change):** Adding significant new features, such as the AI recommendation engine or allowing researchers to upload their own datasets.

2. **Action:** This triggers a full review of the risk management cycle (Sections 1 through 6). New assets (e.g., "S3 bucket for user-uploaded data") must be identified, new threats (e.g., "Malicious file upload") assessed, and new controls (e.g., "Virus scanning on S3 upload") implemented.

3. **Trigger (Minor Change):** A new vulnerability is discovered by the DevSecOps pipeline, or a new threat is reported externally.
4. **Action:** Update the risk register (Section 3) with the new threat/vulnerability and document its treatment.
5. **Periodic Review:** The entire risk assessment should be formally reviewed on a fixed schedule (e.g., annually) or after any significant security incident.

This report detailed a full ISO 27001 risk management cycle, transforming the project's vulnerable initial architecture into a robust, secure design. The resulting treatment plan is proactive, embedding "Defense in Depth," cloud (ISO 27017), and application (ISO 27034) security controls directly into the DevSecOps pipeline.

By addressing both current challenges and emerging AI-specific risks , this process provides the foundation for a resilient system, ensured by a continuous monitoring and review strategy.